Jarvis

Complete project source

Generated from the current project files.

- README.md
- index.html
- styles.css
- app.js
- main.py
- requirements.txt

README.md

```
# Jarvis — AI Chat Demo

A responsive chatbot interface with optional account sign-up/sign-in and scripted demo replies. You can chat as a guest; after three messages Jarvis offers an optional sign-in. FastAPI and SQLite provide local account support; chat responses are not generated by a live AI model.

## Run it

### Open in Chrome

- Easiest: open this folder in File Explorer and double-click `index.html`; choose Google Chrome if asked.
- In VS Code: if the Live Server extension is installed, right-click `index.html` and choose **Open with Live Server**.

Both options let you use the scripted guest chat. Chat replies are demo responses, not from a live AI model. Optional account sign-in requires the FastAPI server below.

The page is branded as **Jarvis** and asks search engines not to index it. Running it from your computer keeps it on your computer; do not publish it to a public hosting service if it should remain private. The `noindex` setting is not a security feature and does not prevent access to a publicly hosted page.

### Run the account-enabled server

1. Install Python 3.10 or newer.
2. From this folder, install the dependencies: `python -m pip install -r requirements.txt`
3. Start the server: `python -m uvicorn main:app --host 127.0.0.1 --port 8000`
4. Open `http://127.0.0.1:8000` in Chrome and start chatting. You can stay in guest mode or sign in/create an account from the prompt after three messages. Passwords must be at least 12 characters.

The SQLite database is created as `jarvis.sqlite3` in this folder by default. Set `JARVIS_DATABASE` to use a different database path. For HTTPS deployments, set `JARVIS_COOKIE_SECURE=true`; do not expose this development setup directly to the public internet. If you open `index.html` directly or use a static file server such as Live Server, the browser uses the scripted demo replies; account sign-in requires the FastAPI server.

## Demo topics

Jarvis recognizes greetings, writing-prompt requests, day-planning requests, space questions, fun-fact requests, help, thanks, and goodbyes. Other messages receive a response explaining that the demo is not connected to a live AI model.

## Controls

- Press **Enter** to send a message; use **Shift + Enter** for a new line.
- Select a suggestion card to try an example prompt.
- Select **New chat** or press **Ctrl/Cmd + K** to reset the conversation.
- Select **About demo** for a reminder that replies are scripted and messages are not sent to an AI service.

## Authentication notes

Passwords are stored as salted PBKDF2-HMAC-SHA256 hashes, never as plaintext. Sign-in uses a random, HttpOnly, SameSite=Strict cookie; only a hash of the session token is stored in SQLite. The app checks same-origin authentication requests, and authenticated sessions expire after seven days. Signing in is optional; guest chat remains available.

This is a local learning project, not a production-ready public account service. Before deploying publicly, add HTTPS, login rate limiting, email verification, password reset and recovery, operational monitoring, and a managed database with backups.
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="theme-color" content="#f5f6fa">
    <meta name="description" content="A friendly AI chatbot demo you can run in your browser.">
    <meta name="robots" content="noindex, nofollow">
    <title>Jarvis — AI Chat Demo</title>
    <link rel="stylesheet" href="styles.css">
    <script src="app.js" defer></script>
  </head>
  <body>
    <main class="auth-screen" id="auth-screen" hidden>
      <section class="auth-card" aria-labelledby="auth-title">
        <button class="auth-back" id="auth-back" type="button">← Back to chat</button>
        <a class="brand auth-brand" href="#" aria-label="Jarvis home">
          <span class="brand-mark" aria-hidden="true">✳</span>
          <span>jarvis</span>
        </a>
        <p class="eyebrow">YOUR AI SIDEKICK</p>
        <h1 id="auth-title">Welcome back.</h1>
        <p class="auth-description" id="auth-description">Sign in to continue to your Jarvis workspace.</p>
        <form class="auth-form" id="auth-form">
          <label for="auth-email">Email address</label>
          <input id="auth-email" name="email" type="email" autocomplete="email" maxlength="254" placeholder="you@example.com" required>
          <label for="auth-password">Password</label>
          <input id="auth-password" name="password" type="password" autocomplete="current-password" minlength="12" maxlength="128" placeholder="At least 12 characters" required>
          <p class="auth-error" id="auth-error" role="alert" hidden></p>
          <button class="auth-submit" id="auth-submit" type="submit">Sign in</button>
        </form>
        <p class="auth-switch"><span id="auth-switch-copy">New to Jarvis?</span> <button id="auth-switch" type="button">Create an account</button></p>
        <p class="auth-note">Creating an account is optional. Your demo chat works without signing in.</p>
      </section>
    </main>

    <div class="app-shell" id="app-shell">
      <aside class="sidebar" aria-label="Chat navigation">
        <a class="brand" href="#" aria-label="Jarvis home">
          <span class="brand-mark" aria-hidden="true">✳</span>
          <span>jarvis</span>
        </a>

        <button class="new-chat-button" id="new-chat" type="button">
          <span aria-hidden="true">+</span>
          New chat
          <span class="shortcut">Ctrl / ⌘ K</span>
        </button>

        <div class="sidebar-label">YOUR SPACE</div>
        <button class="history-item is-active" type="button" aria-current="page">
          <span aria-hidden="true">○</span>
          <span id="chat-title">A fresh start</span>
        </button>

        <div class="sidebar-bottom">
          <div class="profile-avatar" aria-hidden="true">Y</div>
          <div class="profile-copy">
            <strong id="profile-email">Guest mode</strong>
            <span>Personal plan</span>
          </div>
          <button class="logout-button" id="logout" type="button" hidden>Sign out</button>
        </div>
```

```
      </aside>

      <main class="main-content">
        <header class="topbar">
          <div class="mobile-brand">
            <span class="brand-mark" aria-hidden="true">✳</span>
            <span>jarvis</span>
          </div>
          <div class="model-picker" aria-label="Chat mode">
            <span class="status-dot" aria-hidden="true"></span>
            Jarvis Demo
            <span class="chevron" aria-hidden="true">⌄</span>
          </div>
          <div class="topbar-actions">
            <button class="account-button" id="account-button" type="button">Sign in</button>
            <button class="share-button" id="about-demo" type="button">
              <span aria-hidden="true">ⓘ</span>
              <span>About demo</span>
            </button>
          </div>
        </header>

        <section class="conversation" aria-label="Conversation">
          <div class="welcome" id="welcome">
            <div class="welcome-orbit" aria-hidden="true">
              <span class="orbit-spark">✳</span>
              <span class="orbit-dot"></span>
            </div>
            <p class="eyebrow">YOUR AI SIDEKICK</p>
            <h1>A little help goes<br> a <span>long way.</span></h1>
            <p class="welcome-description">Ask a question, explore an idea, or just say hello.<br
class="desktop-break"> What would you like to do today?</p>

            <div class="suggestions" aria-label="Suggested prompts">
              <button class="suggestion" type="button" data-prompt="Give me a creative writing prompt">
                <span class="suggestion-icon lavender" aria-hidden="true">✎</span>
                <span><strong>Get creative</strong><small>Give me a writing prompt</small></span>
                <span class="suggestion-arrow" aria-hidden="true">↗</span>
              </button>
              <button class="suggestion" type="button" data-prompt="Help me plan my day">
                <span class="suggestion-icon peach" aria-hidden="true">◷</span>
                <span><strong>Plan my day</strong><small>Make a simple daily plan</small></span>
                <span class="suggestion-arrow" aria-hidden="true">↗</span>
              </button>
              <button class="suggestion" type="button" data-prompt="Explain something interesting about space">
                <span class="suggestion-icon mint" aria-hidden="true">✧</span>
                <span><strong>Learn something</strong><small>Tell me about space</small></span>
                <span class="suggestion-arrow" aria-hidden="true">↗</span>
              </button>
              <button class="suggestion" type="button" data-prompt="Share a fun fact">
                <span class="suggestion-icon butter" aria-hidden="true">♡</span>
                <span><strong>Surprise me</strong><small>Share a fun fact</small></span>
                <span class="suggestion-arrow" aria-hidden="true">↗</span>
              </button>
            </div>
          </div>

          <div class="messages" id="messages" aria-live="polite" aria-label="Chat messages"></div>
        </section>

        <footer class="composer-area">
          <form class="composer" id="chat-form">
            <label class="sr-only" for="message-input">Message Jarvis</label>
            <textarea id="message-input" name="message" rows="1" placeholder="Message Jarvis..."
maxlength="2000" required></textarea>
            <div class="composer-toolbar">
              <span class="demo-hint"><span class="hint-spark" aria-hidden="true">✳</span> Demo replies are
scripted</span>
              <div class="composer-actions">
```

```
                <span class="key-hint">↵</span>
                <button class="send-button" id="send-button" type="submit" aria-label="Send message" disabled>
                  <span aria-hidden="true">↑</span>
                </button>
              </div>
            </div>
          </form>
          <p class="disclaimer">Jarvis can make mistakes. This demo doesn't use a live AI model.</p>
        </footer>
      </main>
    </div>
    <div class="login-prompt" id="login-prompt" role="dialog" aria-modal="true" aria-labelledby="login-prompt-title" aria-describedby="login-prompt-description" hidden>
      <section class="login-prompt-card">
        <button class="prompt-close" id="prompt-close" type="button" aria-label="Close sign-in prompt">×</button>
        <div class="prompt-icon" aria-hidden="true">✳</div>
        <p class="eyebrow">KEEP YOUR JARVIS HANDY</p>
        <h2 id="login-prompt-title">Want to make it yours?</h2>
        <p id="login-prompt-description">Create an account or sign in. It's completely optional—you can keep chatting as a guest.</p>
        <button class="auth-submit" id="prompt-sign-in" type="button">Sign in or create account</button>
        <button class="prompt-continue" id="prompt-continue" type="button">Continue without an account</button>
      </section>
    </div>
    <div class="toast" id="toast" role="status" aria-live="polite"></div>
  </body>
</html>
```

styles.css

```css
@import url("https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Manrope:wght@400;500;600;700;800&display=swap");

:root {
  color-scheme: light;
  --ink: #252432;
  --muted: #898896;
  --line: #ecebf0;
  --purple: #7862df;
  --purple-dark: #6149d1;
  --paper: #fff;
  --canvas: #faf9fc;
  font-family: "DM Sans", sans-serif;
}

* { box-sizing: border-box; }

body {
  min-width: 320px;
  min-height: 100vh;
  margin: 0;
  color: var(--ink);
  background: var(--canvas);
}

button, textarea { font: inherit; }
button { color: inherit; }
[hidden] { display: none !important; }

.auth-screen {
  position: fixed;
  inset: 0;
  z-index: 5;
  display: grid;
  min-height: 100dvh;
  place-items: center;
  padding: 28px 18px;
  background:
    radial-gradient(ellipse at 50% 36%, #f5f0ff 0, #fbfaff 34%, transparent 68%),
    #faf9fc;
}

.auth-card {
  position: relative;
  width: min(100%, 410px);
  padding: 34px 36px 29px;
  border: 1px solid #eeebf3;
  border-radius: 17px;
  background: #fff;
  box-shadow: 0 16px 55px #332c4b0d;
  animation: appear .35s ease both;
}

.auth-back { position: absolute; top: 16px; left: 18px; padding: 5px 0; border: 0; background: transparent; color: #777482; cursor: pointer; font-size: 11px; }
.auth-back:hover { color: var(--purple-dark); }
.auth-brand { margin: 26px 0 28px; padding: 0; }
.auth-card .eyebrow { margin-bottom: 9px; }
.auth-card h1 { font-size: 32px; letter-spacing: -1.5px; }
.auth-description { margin: 10px 0 24px; color: #85828f; font-size: 12px; line-height: 1.6; }
.auth-form { display: grid; gap: 9px; }
.auth-form label { margin-top: 5px; color: #4a4855; font-size: 11px; font-weight: 600; }
.auth-form input { width: 100%; padding: 11px 12px; border: 1px solid #e8e6ed; border-radius: 8px; outline: none; background: white; color: var(--ink); font-size: 12px; transition: border-color .2s, box-shadow .2s; }
.auth-form input:focus { border-color: #bfb3e8; box-shadow: 0 0 0 3px #7862df16; }
.auth-form input::placeholder { color: #aaa8b2; }
```

```
.auth-submit { min-height: 42px; margin-top: 8px; border: 0; border-radius: 8px; background: var(--purple); color: white; cursor: pointer; font-size: 12px; font-weight: 600; transition: background .2s, opacity .2s; }
.auth-submit:hover:not(:disabled) { background: var(--purple-dark); }
.auth-submit:disabled { cursor: wait; opacity: .65; }
.auth-error { margin: 3px 0 0; color: #b34450; font-size: 11px; line-height: 1.5; }
.auth-switch { margin: 19px 0 0; color: #85828f; text-align: center; font-size: 11px; }
.auth-switch button { padding: 0; border: 0; background: transparent; color: var(--purple-dark); cursor: pointer; font-size: inherit; font-weight: 600; }
.auth-note { margin: 21px 0 0; padding-top: 15px; border-top: 1px solid #f0eef3; color: #9c99a5; text-align: center; font-size: 9px; line-height: 1.6; }

.app-shell {
  display: flex;
  width: min(100%, 1600px);
  min-height: 100vh;
  margin: 0 auto;
  background: var(--paper);
}

.sidebar {
  position: relative;
  display: flex;
  flex: 0 0 254px;
  flex-direction: column;
  min-height: 100vh;
  padding: 27px 17px 18px;
  border-right: 1px solid var(--line);
  background: #fcfbfd;
}

.brand, .mobile-brand {
  display: flex;
  align-items: center;
  gap: 9px;
  color: #292738;
  font-family: "Manrope", sans-serif;
  font-size: 22px;
  font-weight: 800;
  letter-spacing: -1.2px;
  text-decoration: none;
}

.brand { padding: 0 9px; }
.brand-mark { color: var(--purple); font-size: 27px; line-height: 1; }

.new-chat-button {
  display: flex;
  align-items: center;
  gap: 10px;
  width: 100%;
  margin: 39px 0 34px;
  padding: 11px 12px;
  border: 1px solid #e8e5ee;
  border-radius: 9px;
  background: white;
  box-shadow: 0 2px 6px #28223b08;
  cursor: pointer;
  font-size: 13px;
  font-weight: 600;
  transition: border-color .2s, box-shadow .2s, transform .2s;
}

.new-chat-button:hover { border-color: #c9c0eb; box-shadow: 0 3px 12px #28223b0d; transform: translateY(-1px); }
.new-chat-button > span:first-child { color: var(--purple); font-size: 18px; font-weight: 400; }
.shortcut { margin-left: auto; color: #aaa8b3; font-size: 10px; font-weight: 500; }
.sidebar-label { padding: 0 10px; color: #aaa8b3; font-size: 9px; font-weight: 700; letter-spacing: 1.1px; }

.history-item {
  display: flex;
```

```
  align-items: center;
  gap: 11px;
  margin-top: 12px;
  padding: 10px;
  overflow: hidden;
  border: 0;
  border-radius: 8px;
  background: #f1eff8;
  cursor: pointer;
  text-align: left;
  font-size: 12px;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.history-item > span:first-child { color: var(--purple); font-size: 16px; }
.sidebar-bottom { display: flex; align-items: center; gap: 10px; margin-top: auto; padding: 15px 3px 1px; border-top: 1px solid var(--line); }
.profile-avatar { display: grid; width: 32px; height: 32px; place-items: center; border-radius: 50%; background: #f0e9ff; color: #7155c8; font-size: 12px; font-weight: 700; }
.profile-copy { display: grid; gap: 3px; font-size: 11px; }
.profile-copy strong { font-weight: 600; }
.profile-copy span { color: var(--muted); font-size: 10px; }
.logout-button { margin-left: auto; padding: 6px 8px; border: 1px solid #ecebf0; border-radius: 6px; background: white; color: #777482; cursor: pointer; font-size: 10px; }
.logout-button:hover { border-color: #d7d0f0; color: var(--purple-dark); }

.main-content { position: relative; display: flex; flex: 1; flex-direction: column; min-width: 0; min-height: 100vh; }
.topbar { display: flex; height: 72px; flex: 0 0 72px; align-items: center; justify-content: space-between; padding: 0 38px; border-bottom: 1px solid #f0eff3; }
.topbar-actions { display: flex; align-items: center; gap: 9px; }
.account-button { padding: 8px 11px; border: 1px solid #e8e5ee; border-radius: 7px; background: white; color: var(--purple-dark); cursor: pointer; font-size: 11px; font-weight: 600; transition: background .2s, border-color .2s; }
.account-button:hover { border-color: #d7d0f0; background: #faf9ff; }
.mobile-brand { display: none; font-size: 19px; }
.mobile-brand .brand-mark { font-size: 23px; }
.model-picker { display: flex; align-items: center; gap: 8px; color: #555361; font-size: 12px; font-weight: 600; }
.status-dot { width: 7px; height: 7px; border-radius: 50%; background: #75c9a0; box-shadow: 0 0 0 3px #75c9a01e; }
.chevron { color: #a4a1ad; font-size: 16px; }
.share-button { display: flex; align-items: center; gap: 7px; padding: 8px 11px; border: 1px solid #ecebf0; border-radius: 7px; background: white; color: #656271; cursor: pointer; font-size: 11px; transition: background .2s; }
.share-button:hover { background: #f8f7fa; }

.conversation { display: flex; flex: 1; flex-direction: column; justify-content: center; width: min(100% - 48px, 660px); margin: 0 auto; padding: 30px 0 14px; }
.welcome { width: 100%; margin: auto 0; padding: 24px 0; animation: appear .5s ease both; }
.welcome-orbit { position: relative; display: grid; width: 53px; height: 53px; place-items: center; margin-bottom: 25px; border: 1px solid #eae6f5; border-radius: 17px; background: linear-gradient(145deg, #fff, #f8f5ff); box-shadow: 0 5px 16px #7155c80d; transform: rotate(-6deg); }
.orbit-spark { color: var(--purple); font-size: 28px; }
.orbit-dot { position: absolute; top: 8px; right: 8px; width: 6px; height: 6px; border-radius: 50%; background: #f2b76f; }
.eyebrow { margin: 0 0 11px; color: var(--purple); font-size: 9px; font-weight: 700; letter-spacing: 1.7px; }
h1 { margin: 0; font-family: "Manrope", sans-serif; font-size: clamp(35px, 5vw, 46px); font-weight: 700; letter-spacing: -2.4px; line-height: 1.12; }
h1 span { color: var(--purple); }
.welcome-description { margin: 16px 0 27px; color: #82808d; font-size: 13px; line-height: 1.75; }
.suggestions { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 10px; }
.suggestion { display: flex; min-width: 0; align-items: center; gap: 11px; padding: 12px; border: 1px solid #eeedf2; border-radius: 10px; background: #fff; cursor: pointer; text-align: left; transition: transform .18s, border-color .18s, box-shadow .18s; }
.suggestion:hover { border-color: #d7d0f0; box-shadow: 0 5px 15px #39334f0b; transform: translateY(-2px); }
.suggestion-icon { display: grid; width: 33px; height: 33px; flex: 0 0 33px; place-items: center; border-radius: 9px; font-size: 15px; }
```

```
.lavender { background: #f0edff; color: #806be3; }
.peach { background: #fff1e9; color: #de9365; }
.mint { background: #e9f7f1; color: #54a981; }
.butter { background: #fff7df; color: #d8a747; }
.suggestion > span:nth-child(2) { display: grid; min-width: 0; gap: 4px; }
.suggestion strong { font-size: 11px; font-weight: 600; }
.suggestion small { overflow: hidden; color: #95929e; font-size: 10px; text-overflow: ellipsis; white-space: nowrap; }
.suggestion-arrow { margin-left: auto; align-self: flex-start; color: #b6b3be; font-size: 12px; }

.messages { display: grid; gap: 22px; width: 100%; padding: 14px 0; }
.message { display: flex; align-items: flex-start; gap: 11px; animation: appear .25s ease both; }
.message.user { justify-content: flex-end; }
.assistant-avatar { display: grid; width: 29px; height: 29px; flex: 0 0 29px; place-items: center; border: 1px solid #ece8f7; border-radius: 10px; background: #f8f6ff; color: var(--purple); font-size: 17px; }
.message-body { max-width: min(82%, 500px); }
.message.user .message-body { max-width: min(82%, 480px); }
.message-meta { display: flex; align-items: center; gap: 8px; margin: 1px 0 6px; color: #85828f; font-size: 10px; }
.message-meta strong { color: #4a4855; font-size: 11px; font-weight: 600; }
.message.user .message-meta { justify-content: flex-end; }
.bubble { padding: 11px 14px; border: 1px solid #eeedf1; border-radius: 3px 12px 12px; background: #fff; color: #4f4d5a; font-size: 12px; line-height: 1.7; white-space: pre-wrap; overflow-wrap: anywhere; }
.message.user .bubble { border-color: #7562d7; border-radius: 12px 3px 12px 12px; background: var(--purple); color: white; }
.typing .bubble { display: flex; width: 55px; gap: 4px; padding: 14px; }
.typing-dot { width: 5px; height: 5px; border-radius: 50%; background: #aaa3c1; animation: pulse 1s infinite ease-in-out; }
.typing-dot:nth-child(2) { animation-delay: .15s; }
.typing-dot:nth-child(3) { animation-delay: .3s; }

.composer-area { width: min(100% - 48px, 660px); margin: 0 auto; padding: 6px 0 16px; }
.composer { padding: 11px 12px 9px 16px; border: 1px solid #e8e6ed; border-radius: 13px; background: #fff; box-shadow: 0 5px 19px #28223b08; transition: border-color .2s, box-shadow .2s; }
.composer:focus-within { border-color: #c9bee9; box-shadow: 0 5px 20px #7862df12; }
.composer textarea { display: block; width: 100%; max-height: 150px; min-height: 26px; resize: none; border: 0; outline: 0; background: transparent; color: var(--ink); font-size: 12px; line-height: 1.6; }
.composer textarea::placeholder { color: #aaa8b2; }
.composer-toolbar { display: flex; align-items: center; justify-content: space-between; margin-top: 5px; }
.demo-hint { display: flex; align-items: center; gap: 6px; color: #9c99a5; font-size: 10px; }
.hint-spark { color: var(--purple); font-size: 13px; }
.composer-actions { display: flex; align-items: center; gap: 10px; }
.key-hint { color: #aaa7b2; font-size: 12px; }
.send-button { display: grid; width: 29px; height: 29px; place-items: center; border: 0; border-radius: 8px; background: var(--purple); color: white; cursor: pointer; font-size: 19px; transition: background .2s, opacity .2s, transform .2s; }
.send-button:hover:not(:disabled) { background: var(--purple-dark); transform: translateY(-1px); }
.send-button:disabled { cursor: not-allowed; opacity: .42; }
.disclaimer { margin: 10px 0 0; color: #aaa8b3; text-align: center; font-size: 9px; }
.toast { position: fixed; right: 22px; bottom: 22px; z-index: 3; max-width: calc(100vw - 44px); padding: 11px 15px; border: 1px solid #e8e5ef; border-radius: 9px; background: white; box-shadow: 0 8px 24px #28223b14; color: #555261; font-size: 12px; opacity: 0; pointer-events: none; transform: translateY(7px); transition: opacity .2s, transform .2s; }
.toast.is-visible { opacity: 1; transform: translateY(0); }
.login-prompt { position: fixed; inset: 0; z-index: 4; display: grid; place-items: center; padding: 20px; background: #211b3280; backdrop-filter: blur(4px); animation: appear .2s ease both; }
.login-prompt-card { position: relative; width: min(100%, 390px); padding: 30px 28px 24px; border: 1px solid #eeebf3; border-radius: 16px; background: white; box-shadow: 0 18px 60px #17112630; text-align: center; }
.prompt-close { position: absolute; top: 11px; right: 13px; width: 30px; height: 30px; border: 0; border-radius: 8px; background: transparent; color: #9794a0; cursor: pointer; font-size: 23px; line-height: 1; }
.prompt-close:hover { background: #f5f3f8; color: #4b4856; }
.prompt-icon { display: grid; width: 44px; height: 44px; place-items: center; margin: 0 auto 17px; border: 1px solid #eae6f5; border-radius: 14px; background: #f8f6ff; color: var(--purple); font-size: 24px; }
.login-prompt-card .eyebrow { margin-bottom: 8px; }
.login-prompt-card h2 { margin: 0; font-family: "Manrope", sans-serif; font-size: 22px; letter-spacing: -.8px; }
.login-prompt-card > p:not(.eyebrow) { margin: 11px 0 20px; color: #85828f; font-size: 12px; line-height: 1.65; }
.login-prompt-card .auth-submit { width: 100%; }
```

```
.prompt-continue { width: 100%; margin-top: 11px; padding: 9px 10px; border: 0; border-radius: 7px; background: transparent; color: #777482; cursor: pointer; font-size: 11px; }
.prompt-continue:hover { background: #f8f7fa; color: #4b4856; }
.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; overflow: hidden; clip: rect(0, 0, 0, 0); white-space: nowrap; clip-path: inset(50%); }

@keyframes appear { from { opacity: 0; transform: translateY(7px); } to { opacity: 1; transform: translateY(0); } }
@keyframes pulse { 0%, 60%, 100% { opacity: .4; transform: translateY(0); } 30% { opacity: 1; transform: translateY(-3px); } }

@media (min-height: 800px) {
  .welcome { padding-top: 42px; padding-bottom: 42px; }
}

@media (max-width: 700px) {
  .auth-card { padding: 29px 25px 24px; }
  .sidebar { display: none; }
  .topbar { height: 60px; flex-basis: 60px; padding: 0 19px; }
  .mobile-brand { display: flex; }
  .model-picker { margin-left: auto; margin-right: 14px; font-size: 11px; }
  .share-button { padding: 7px 9px; }
  .share-button span:last-child { display: none; }
  .conversation, .composer-area { width: min(100% - 38px, 540px); }
  .conversation { padding-top: 15px; padding-bottom: 4px; }
  .welcome { padding: 20px 0; }
  .welcome-orbit { margin-bottom: 19px; }
}

@media (max-width: 440px) {
  .topbar { padding: 0 14px; }
  .conversation, .composer-area { width: calc(100% - 28px); }
  h1 { font-size: 38px; }
  .welcome-description { font-size: 12px; }
  .desktop-break { display: none; }
  .suggestions { gap: 8px; }
  .suggestion { gap: 8px; padding: 10px 8px; }
  .suggestion-icon { width: 29px; height: 29px; flex-basis: 29px; }
  .suggestion strong { font-size: 10px; }
  .suggestion small { font-size: 9px; }
  .demo-hint { font-size: 9px; }
  .login-prompt-card { padding: 27px 20px 19px; }
}

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { scroll-behavior: auto !important; animation-duration: .01ms !important; animation-iteration-count: 1 !important; transition-duration: .01ms !important; }
}
```

app.js

```
const form = document.querySelector("#chat-form");
const input = document.querySelector("#message-input");
const sendButton = document.querySelector("#send-button");
const messages = document.querySelector("#messages");
const welcome = document.querySelector("#welcome");
const newChatButton = document.querySelector("#new-chat");
const chatTitle = document.querySelector("#chat-title");
const toast = document.querySelector("#toast");
const authScreen = document.querySelector("#auth-screen");
const appShell = document.querySelector("#app-shell");
const authForm = document.querySelector("#auth-form");
const authEmail = document.querySelector("#auth-email");
const authPassword = document.querySelector("#auth-password");
const authError = document.querySelector("#auth-error");
const authTitle = document.querySelector("#auth-title");
const authDescription = document.querySelector("#auth-description");
const authSubmit = document.querySelector("#auth-submit");
const authSwitchCopy = document.querySelector("#auth-switch-copy");
const authSwitch = document.querySelector("#auth-switch");
const profileEmail = document.querySelector("#profile-email");
const logoutButton = document.querySelector("#logout");
const accountButton = document.querySelector("#account-button");
const loginPrompt = document.querySelector("#login-prompt");

let toastTimeout;
let authMode = "login";
let chatCount = 0;
let loginPromptShown = false;
let authenticated = false;

function showToast(message) {
  toast.textContent = message;
  toast.classList.add("is-visible");
  window.clearTimeout(toastTimeout);
  toastTimeout = window.setTimeout(() => toast.classList.remove("is-visible"), 3200);
}

function formatTime(date = new Date()) {
  return new Intl.DateTimeFormat(undefined, { hour: "numeric", minute: "2-digit" }).format(date);
}

async function apiRequest(path, options = {}) {
  let response;
  try {
    response = await fetch(path, {
      ...options,
      credentials: "same-origin",
      headers: {
        ...(options.body ? { "Content-Type": "application/json" } : {}),
        ...options.headers,
      },
    });
  } catch {
    const error = new Error("Can't reach Jarvis.");
    error.code = "NETWORK_UNAVAILABLE";
    throw error;
  }

  const data = await response.json().catch(() => ({}));
  if (!response.ok) {
    const error = new Error(data.detail || "Something went wrong. Please try again.");
    error.status = response.status;
    throw error;
  }
  return data;
}
```

```
67
68 function showAuthError(message) {
69   authError.textContent = message;
70   authError.hidden = !message;
71 }
72
73 function setAuthenticated(user) {
74   authenticated = true;
75   profileEmail.textContent = user.email;
76   logoutButton.hidden = false;
77   accountButton.hidden = true;
78   authScreen.hidden = true;
79   input.focus();
80 }
81
82 function setUnauthenticated() {
83   authenticated = false;
84   profileEmail.textContent = "Guest mode";
85   logoutButton.hidden = true;
86   accountButton.hidden = false;
87 }
88
89 function openAuthScreen() {
90   loginPrompt.hidden = true;
91   authScreen.hidden = false;
92   authPassword.value = "";
93   authEmail.value = "";
94   authMode = "login";
95   updateAuthMode();
96   authEmail.focus();
97 }
98
99 function closeLoginPrompt() {
100   loginPrompt.hidden = true;
101   loginPromptShown = true;
102 }
103
104 function maybeShowLoginPrompt() {
105   if (chatCount >= 3 && !loginPromptShown && !authenticated) {
106     loginPrompt.hidden = false;
107     loginPromptShown = true;
108     document.querySelector("#prompt-close").focus();
109   }
110 }
111
112 function updateAuthMode() {
113   const creatingAccount = authMode === "signup";
114   authTitle.textContent = creatingAccount ? "Create your account." : "Welcome back.";
115   authDescription.textContent = creatingAccount
116     ? "Create an account to start using your Jarvis workspace."
117     : "Sign in to continue to your Jarvis workspace.";
118   authSubmit.textContent = creatingAccount ? "Create account" : "Sign in";
119   authSwitchCopy.textContent = creatingAccount ? "Already have an account?" : "New to Jarvis?";
120   authSwitch.textContent = creatingAccount ? "Sign in" : "Create an account";
121   authPassword.autocomplete = creatingAccount ? "new-password" : "current-password";
122   showAuthError("");
123 }
124
125 function addMessage(role, text, { typing = false } = {}) {
126   const article = document.createElement("article");
127   article.className = `message ${role}${typing ? " typing" : ""}`;
128
129   if (role === "assistant") {
130     const avatar = document.createElement("div");
131     avatar.className = "assistant-avatar";
132     avatar.setAttribute("aria-hidden", "true");
133     avatar.textContent = "✳";
134     article.append(avatar);
135   }
```

```

  const body = document.createElement("div");
  body.className = "message-body";
  const meta = document.createElement("div");
  meta.className = "message-meta";
  const name = document.createElement("strong");
  name.textContent = role === "user" ? "You" : "Jarvis";
  const time = document.createElement("span");
  time.textContent = formatTime();
  meta.append(name, time);

  const bubble = document.createElement("div");
  bubble.className = "bubble";
  if (typing) {
    bubble.setAttribute("aria-label", "Jarvis is typing");
    for (let index = 0; index < 3; index += 1) {
      const dot = document.createElement("span");
      dot.className = "typing-dot";
      dot.setAttribute("aria-hidden", "true");
      bubble.append(dot);
    }
  } else {
    bubble.textContent = text;
  }

  body.append(meta, bubble);
  article.append(body);
  messages.append(article);
  article.scrollIntoView({ behavior: "smooth", block: "nearest" });
  return article;
}

function getDemoReply(message) {
  const text = message.toLowerCase();
  if (/\b(hi|hello|hey|good morning|good afternoon|good evening)\b/.test(text)) {
    return "Hi there! I'm Jarvis, a chatbot demo. I can share a fun fact, suggest a writing prompt, help sketch out a simple plan, or chat about space. What sounds good?";
  }
  if (/\b(write|writing|creative|story|prompt|poem)\b/.test(text)) {
    return "Here's a writing prompt: A small bookstore receives a book that describes tomorrow in perfect detail—but the final page is blank. What happens next?";
  }
  if (/\b(plan|schedule|day|organize|organise|today|week)\b/.test(text)) {
    return "Try this simple plan:\n\n1. Pick your top priority and give it a focused 45-minute block.\n2. Take a short break, then handle one quick task.\n3. Leave some room for lunch and a walk.\n4. Wrap up by noting one thing to start with tomorrow.\n\nTell me what you need to fit in and I can help shape the plan.";
  }
  if (/\b(space|planet|star|moon|universe|astronomy)\b/.test(text)) {
    return "Here's a space fact: a day on Venus is longer than its year. Venus takes about 243 Earth days to rotate once, but only about 225 Earth days to orbit the Sun.";
  }
  if (/\b(fun fact|surprise|interesting|fact)\b/.test(text)) {
    return "Fun fact: octopuses have three hearts. Two pump blood to the gills, while the third sends it to the rest of the body.";
  }
  if (/\b(help|what can you do|capabilities)\b/.test(text)) {
    return "I can respond to a few demo topics: greetings, creative writing prompts, simple day planning, space facts, and fun facts. Try one of those—or ask me something else to see the demo fallback.";
  }
  if (/\b(thank|thanks|thx)\b/.test(text)) {
    return "You're welcome! This is a scripted demo, so my replies are limited—but you can keep exploring the example prompts.";
  }
  if (/\b(bye|goodbye|see you)\b/.test(text)) {
    return "See you next time! Start a new chat whenever you'd like to explore another prompt.";
  }
  return "That's an interesting question. I'm running in demo mode, so I don't have a live AI model connected yet. Try asking for a writing prompt, a simple day plan, a space fact, or a fun fact—or connect an AI API to make Jarvis answer freely.";
```

```
}

function updateInputState() {
  sendButton.disabled = input.value.trim().length === 0;
  input.style.height = "auto";
  input.style.height = `${Math.min(input.scrollHeight, 150)}px`;
}

async function sendMessage(text) {
  const message = text.trim();
  if (!message) return;

  welcome.hidden = true;
  if (messages.childElementCount === 0) {
    chatTitle.textContent = message.length > 25 ? `${message.slice(0, 25)}…` : message;
  }
  addMessage("user", message);
  chatCount += 1;
  input.value = "";
  updateInputState();

  const typing = addMessage("assistant", "", { typing: true });
  try {
    const data = window.location.protocol === "file:"
      ? { reply: getDemoReply(message) }
      : await apiRequest("/api/chat", {
        method: "POST",
        body: JSON.stringify({ message }),
      });
    typing.remove();
    addMessage("assistant", data.reply);
  } catch (error) {
    typing.remove();
    if (error.code === "NETWORK_UNAVAILABLE" || error.status === 404) {
      addMessage("assistant", getDemoReply(message));
    } else {
      addMessage("assistant", error.message);
    }
  }
  maybeShowLoginPrompt();
}

form.addEventListener("submit", (event) => {
  event.preventDefault();
  sendMessage(input.value);
  input.focus();
});

input.addEventListener("input", updateInputState);
input.addEventListener("keydown", (event) => {
  if (event.key === "Enter" && !event.shiftKey) {
    event.preventDefault();
    form.requestSubmit();
  }
});

document.querySelectorAll(".suggestion").forEach((button) => {
  button.addEventListener("click", () => sendMessage(button.dataset.prompt || ""));
});

newChatButton.addEventListener("click", () => {
  messages.replaceChildren();
  welcome.hidden = false;
  chatTitle.textContent = "A fresh start";
  input.value = "";
  updateInputState();
  input.focus();
});

```

```
document.querySelector("#about-demo").addEventListener("click", () => {
  showToast("Jarvis uses scripted example replies. No messages are sent to an AI service.");
});

accountButton.addEventListener("click", openAuthScreen);
document.querySelector("#auth-back").addEventListener("click", () => {
  authScreen.hidden = true;
  showAuthError("");
  input.focus();
});
document.querySelector("#prompt-sign-in").addEventListener("click", openAuthScreen);
document.querySelector("#prompt-close").addEventListener("click", closeLoginPrompt);
document.querySelector("#prompt-continue").addEventListener("click", closeLoginPrompt);
loginPrompt.addEventListener("click", (event) => {
  if (event.target === loginPrompt) closeLoginPrompt();
});
document.addEventListener("keydown", (event) => {
  if (event.key === "Escape" && !loginPrompt.hidden) closeLoginPrompt();
});

document.addEventListener("keydown", (event) => {
  if ((event.metaKey || event.ctrlKey) && event.key.toLowerCase() === "k") {
    event.preventDefault();
    newChatButton.click();
  }
});

authSwitch.addEventListener("click", () => {
  authMode = authMode === "login" ? "signup" : "login";
  updateAuthMode();
});

authForm.addEventListener("submit", async (event) => {
  event.preventDefault();
  showAuthError("");
  authSubmit.disabled = true;
  try {
    if (window.location.protocol === "file:") {
      throw new Error("Account sign-in needs the FastAPI server. See README.md for setup; guest chat works without it.");
    }
    const user = await apiRequest(`/api/auth/${authMode}`, {
      method: "POST",
      body: JSON.stringify({
        email: authEmail.value,
        password: authPassword.value,
      }),
    });
    authPassword.value = "";
    setAuthenticated(user);
    showToast("You're signed in to Jarvis.");
  } catch (error) {
    showAuthError(error.message);
  } finally {
    authSubmit.disabled = false;
  }
});

document.querySelector("#logout").addEventListener("click", async () => {
  try {
    await apiRequest("/api/auth/logout", { method: "POST" });
    setUnauthenticated();
    showToast("Signed out. You can keep chatting as a guest.");
  } catch (error) {
    showToast(error.message);
  }
});

async function restoreSession() {
```

```
  try {
    const user = await apiRequest("/api/me");
    if (user.authenticated) {
      setAuthenticated(user);
    } else {
      setUnauthenticated();
    }
  } catch {
    setUnauthenticated();
  }
}

updateAuthMode();
restoreSession();

```

main.py

```python
import hashlib
import hmac
import os
import re
import secrets
import sqlite3
import time
from contextlib import asynccontextmanager, contextmanager
from pathlib import Path
from typing import AsyncIterator, Iterator
from urllib.parse import urlsplit

from fastapi import FastAPI, HTTPException, Request, Response
from fastapi.responses import FileResponse


BASE_DIR = Path(__file__).resolve().parent
DATABASE_PATH = Path(os.environ.get("JARVIS_DATABASE", BASE_DIR / "jarvis.sqlite3"))
SESSION_COOKIE = "jarvis_session"
SESSION_DURATION_SECONDS = 7 * 24 * 60 * 60
PASSWORD_ITERATIONS = 600_000
COOKIE_SECURE = os.environ.get("JARVIS_COOKIE_SECURE", "false").lower() == "true"
EMAIL_PATTERN = re.compile(r"[^@\s]{1,64}@[^@\s.]+(?:\.[^@\s.]+)+")


@contextmanager
def database() -> Iterator[sqlite3.Connection]:
    DATABASE_PATH.parent.mkdir(parents=True, exist_ok=True)
    connection = sqlite3.connect(DATABASE_PATH, timeout=10)
    connection.row_factory = sqlite3.Row
    try:
        yield connection
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    finally:
        connection.close()


def initialize_database() -> None:
    with database() as connection:
        connection.executescript(
            """
            CREATE TABLE IF NOT EXISTS users (
                id INTEGER PRIMARY KEY,
                email TEXT NOT NULL UNIQUE,
                salt TEXT NOT NULL,
                password_hash TEXT NOT NULL,
                created_at INTEGER NOT NULL
            );
            CREATE TABLE IF NOT EXISTS sessions (
                token_hash TEXT PRIMARY KEY,
                user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
                expires_at INTEGER NOT NULL
            );
            CREATE INDEX IF NOT EXISTS sessions_user_id_idx ON sessions(user_id);
            DELETE FROM sessions WHERE expires_at <= strftime('%s', 'now');
            """
        )


def hash_password(password: str, salt: bytes) -> bytes:
    return hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, PASSWORD_ITERATIONS
    )
```

```
def validate_credentials(payload: object) -> tuple[str, str]:
    if not isinstance(payload, dict):
        raise HTTPException(status_code=400, detail="Invalid request.")

    email = payload.get("email")
    password = payload.get("password")
    if not isinstance(email, str) or not isinstance(password, str):
        raise HTTPException(status_code=400, detail="Email and password are required.")

    email = email.strip().lower()
    if len(email) > 254 or not EMAIL_PATTERN.fullmatch(email):
        raise HTTPException(status_code=400, detail="Enter a valid email address.")
    if len(password) < 12:
        raise HTTPException(
            status_code=400, detail="Password must be at least 12 characters."
        )
    if len(password) > 128:
        raise HTTPException(
            status_code=400, detail="Password must be 128 characters or fewer."
        )
    return email, password


def validate_same_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if not origin:
        return
    parsed_origin = urlsplit(origin)
    request_origin = urlsplit(str(request.base_url))
    if (parsed_origin.scheme, parsed_origin.netloc) != (
        request_origin.scheme,
        request_origin.netloc,
    ):
        raise HTTPException(status_code=403, detail="Cross-origin request rejected.")


def issue_session(response: Response, user_id: int) -> None:
    raw_token = secrets.token_urlsafe(32)
    token_hash = hashlib.sha256(raw_token.encode("ascii")).hexdigest()
    expires_at = int(time.time()) + SESSION_DURATION_SECONDS
    with database() as connection:
        connection.execute(
            "INSERT INTO sessions (token_hash, user_id, expires_at) VALUES (?, ?, ?)",
            (token_hash, user_id, expires_at),
        )
    response.set_cookie(
        SESSION_COOKIE,
        raw_token,
        max_age=SESSION_DURATION_SECONDS,
        httponly=True,
        secure=COOKIE_SECURE,
        samesite="strict",
        path="/",
    )


def current_user(request: Request) -> sqlite3.Row | None:
    raw_token = request.cookies.get(SESSION_COOKIE)
    if not raw_token:
        return None

    token_hash = hashlib.sha256(raw_token.encode("utf-8")).hexdigest()
    with database() as connection:
        row = connection.execute(
            """
            SELECT users.id, users.email, sessions.expires_at
            FROM sessions JOIN users ON users.id = sessions.user_id
```

```
            WHERE sessions.token_hash = ?
            """,
            (token_hash,),
        ).fetchone()
        if row is not None and row["expires_at"] <= int(time.time()):
            connection.execute(
                "DELETE FROM sessions WHERE token_hash = ?", (token_hash,)
            )
            return None
        return row


def get_demo_reply(message: str) -> str:
    text = message.lower()
    if re.search(r"\b(hi|hello|hey|good morning|good afternoon|good evening)\b", text):
        return (
            "Hi there! I'm Jarvis, a chatbot demo. I can share a fun fact, suggest "
            "a writing prompt, help sketch out a simple plan, or chat about space. "
            "What sounds good?"
        )
    if re.search(r"\b(write|writing|creative|story|prompt|poem)\b", text):
        return (
            "Here's a writing prompt: A small bookstore receives a book that "
            "describes tomorrow in perfect detail—but the final page is blank. "
            "What happens next?"
        )
    if re.search(r"\b(plan|schedule|day|organize|organise|today|week)\b", text):
        return (
            "Try this simple plan:\n\n1. Pick your top priority and give it a focused "
            "45-minute block.\n2. Take a short break, then handle one quick task.\n"
            "3. Leave some room for lunch and a walk.\n4. Wrap up by noting one "
            "thing to start with tomorrow.\n\nTell me what you need to fit in and I "
            "can help shape the plan."
        )
    if re.search(r"\b(space|planet|star|moon|universe|astronomy)\b", text):
        return (
            "Here's a space fact: a day on Venus is longer than its year. Venus "
            "takes about 243 Earth days to rotate once, but only about 225 Earth "
            "days to orbit the Sun."
        )
    if re.search(r"\b(fun fact|surprise|interesting|fact)\b", text):
        return (
            "Fun fact: octopuses have three hearts. Two pump blood to the gills, "
            "while the third sends it to the rest of the body."
        )
    if re.search(r"\b(help|what can you do|capabilities)\b", text):
        return (
            "I can respond to a few demo topics: greetings, creative writing "
            "prompts, simple day planning, space facts, and fun facts. Try one of "
            "those—or ask me something else to see the demo fallback."
        )
    if re.search(r"\b(thank|thanks|thx)\b", text):
        return (
            "You're welcome! This is a scripted demo, so my replies are limited—but "
            "you can keep exploring the example prompts."
        )
    if re.search(r"\b(bye|goodbye|see you)\b", text):
        return "See you next time! Start a new chat whenever you'd like to explore another prompt."
    return (
        "That's an interesting question. I'm running in demo mode, so I don't have "
        "a live AI model connected yet. Try asking for a writing prompt, a simple "
        "day plan, a space fact, or a fun fact—or connect an AI API to make Jarvis "
        "answer freely."
    )


@asynccontextmanager
async def lifespan(_: FastAPI) -> AsyncIterator[None]:
    initialize_database()
```

```
    yield


app = FastAPI(
    lifespan=lifespan,
    docs_url=None,
    redoc_url=None,
    openapi_url=None,
)


@app.middleware("http")
async def set_api_cache_headers(request: Request, call_next):
    response = await call_next(request)
    if request.url.path.startswith("/api/"):
        response.headers["Cache-Control"] = "no-store"
    return response


@app.get("/")
def home() -> FileResponse:
    return FileResponse(BASE_DIR / "index.html")


@app.get("/styles.css")
def stylesheet() -> FileResponse:
    return FileResponse(BASE_DIR / "styles.css", media_type="text/css")


@app.get("/app.js")
def javascript() -> FileResponse:
    return FileResponse(BASE_DIR / "app.js", media_type="text/javascript")


@app.post("/api/auth/signup")
async def signup(request: Request, response: Response) -> dict[str, object]:
    validate_same_origin(request)
    try:
        payload = await request.json()
    except ValueError as error:
        raise HTTPException(status_code=400, detail="Invalid JSON body.") from error
    email, password = validate_credentials(payload)
    salt = secrets.token_bytes(16)
    password_hash = hash_password(password, salt)

    try:
        with database() as connection:
            cursor = connection.execute(
                """
                INSERT INTO users (email, salt, password_hash, created_at)
                VALUES (?, ?, ?, ?)
                """,
                (email, salt.hex(), password_hash.hex(), int(time.time())),
            )
            user_id = cursor.lastrowid
    except sqlite3.IntegrityError as error:
        raise HTTPException(
            status_code=409, detail="An account with that email already exists."
        ) from error

    issue_session(response, user_id)
    return {"authenticated": True, "email": email}


@app.post("/api/auth/login")
async def login(request: Request, response: Response) -> dict[str, object]:
    validate_same_origin(request)
    try:
        payload = await request.json()
```

```
    except ValueError as error:
        raise HTTPException(status_code=400, detail="Invalid JSON body.") from error
    email, password = validate_credentials(payload)

    with database() as connection:
        user = connection.execute(
            "SELECT id, email, salt, password_hash FROM users WHERE email = ?",
            (email,),
        ).fetchone()

    salt = bytes.fromhex(user["salt"]) if user else bytes(16)
    stored_hash = bytes.fromhex(user["password_hash"]) if user else bytes(32)
    password_matches = hmac.compare_digest(hash_password(password, salt), stored_hash)
    if user is None or not password_matches:
        raise HTTPException(status_code=401, detail="Email or password is incorrect.")

    issue_session(response, user["id"])
    return {"authenticated": True, "email": user["email"]}


@app.get("/api/me")
def me(request: Request) -> dict[str, object]:
    user = current_user(request)
    if user is None:
        return {"authenticated": False}
    return {"authenticated": True, "email": user["email"]}


@app.post("/api/auth/logout")
def logout(request: Request, response: Response) -> dict[str, bool]:
    validate_same_origin(request)
    raw_token = request.cookies.get(SESSION_COOKIE)
    if raw_token:
        token_hash = hashlib.sha256(raw_token.encode("utf-8")).hexdigest()
        with database() as connection:
            connection.execute(
                "DELETE FROM sessions WHERE token_hash = ?", (token_hash,)
            )
    response.delete_cookie(
        SESSION_COOKIE,
        httponly=True,
        secure=COOKIE_SECURE,
        samesite="strict",
        path="/",
    )
    return {"logged_out": True}


@app.post("/api/chat")
async def chat(request: Request) -> dict[str, str]:
    try:
        payload = await request.json()
    except ValueError as error:
        raise HTTPException(status_code=400, detail="Invalid JSON body.") from error
    if not isinstance(payload, dict):
        raise HTTPException(status_code=400, detail="Invalid request.")
    message = payload.get("message")
    if not isinstance(message, str) or not message.strip():
        raise HTTPException(status_code=400, detail="Enter a message.")
    if len(message) > 2000:
        raise HTTPException(status_code=400, detail="Message must be 2000 characters or fewer.")
    return {"reply": get_demo_reply(message.strip())}

```

requirements.txt

```
fastapi>=0.115,<1.0
uvicorn>=0.30,<1.0

```